FEM Basics

Oliver Jackson

October 17, 2026

Contents

1 Introduction **1**

1.1 The weak form 1

1.2 Weak form over a finite number of nodes 2

1.3 General algorithm for stiffness matrix construction 4

1.4 Neumann and Robin Boundary Conditions 6

2 Programming **8**

2.1 File structure 8

2.2 main.py 9

2.3 build_stiffness_matrices.py 10

2.4 basis.py 12

2.5 gaussian_quadrature.py 13

2.6 information_matrices.py 14

2.7 functions.py 15

2.8 boundary_conditions.py and error.py 15

Chapter 1

Introduction

This document details the FEM Galerkin method. I included both the derivation of the method and am developing a software package to implement the method in Python.

1.1 The weak form

We are considering problems of the ellipcic type in one dimension.

$$-\frac{d}{dx}\left(c(x)\frac{du}{dx}\right) = f(x) \quad \text{with solution } u.$$

The space $c^k(\bar{\Omega})$ is defined to be the set of all real-valved functions a defined on $\bar{\Omega}$ (which is both Ω and $\partial\Omega$) with the property that u and its derivatives up to the kth order are all continuous on $\bar{\Omega}$. A solution to u is sought in the subspace

$$C_D^2(\bar{\Omega}) = \left\{u \in C^2(\bar{\Omega}) : u = 0 \text{ on } \partial\Omega\right\}.$$

Introduce the test function v and the space of test functions to be $C_D^2(\bar{\Omega})$.

$$-\nabla(c \cdot \nabla u) = f$$

$$\int -\nabla(c \cdot \nabla u) \cdot v = \int f \cdot v.$$

This equation holds for some $u \in C_D^2(\bar{\Omega})$ denoted u_h and for all $v \in C_D^2(\bar{\Omega})$ if and only if u satisfies the original differential equation. The next step is

to apply green's identity to the left hand side. A quick review of Green's identity. The weak form of a BVP is derived using greens identity, which is the multi-dimensional form of integration by parts.

$$\nabla \cdot (v\nabla u) = \nabla v \cdot \nabla u + v\Delta u.$$

Then to obtain Greens identity integrate both sides over Ω and apply the divergence theorem

$$\begin{aligned}
&\Rightarrow \int_\Omega \nabla \cdot (v\nabla u) = \int_\Omega \nabla v \cdot \nabla u + \int_\Omega v\Delta u \\
&\Rightarrow \int_{\partial\Omega} v\nabla u \cdot n = \int_\Omega \nabla v \cdot \nabla u + \int_\Omega v\Delta u \\
&\Rightarrow -\int_\Omega v\Delta u = \int_\Omega \nabla v \cdot \nabla u - \int_{\partial\Omega} v\frac{\partial u}{\partial n}.
\end{aligned}$$

we know $V = 0$ on $\partial\Omega$ so

$$-\int_\Omega v\Delta u = \int_\Omega \nabla V \cdot \nabla u.$$

So we get

$$\int_\Omega -\nabla(c \cdot \nabla u) \cdot v = \int_\Omega c\nabla u \cdot \nabla v$$

So the weak form of our elliptic problem is given by Find $u \in C_D^2(\bar{\Omega})$ such that $\int_\Omega c\nabla u \cdot \nabla v = \int_\Omega fv$ for all $v \in C_D^2(\bar{\Omega})$. We introduce notation $(u, v) = \int_\Omega u \cdot v$ and bilinear operator a,

$$a(u, v) = (f, v) \quad \forall v \in C_D^2(\bar{\Omega})$$

Where $a(u, v) \equiv \int_\Omega c\nabla u \cdot \nabla v$.

1.2 Weak form over a finite number of nodes

Define nodes on the interval $x_0 < x_1 < \cdots < x_N < x_{N+1}$. The space between the nodes N are $h = x_n - x_{n-1}$ where $n = 0, \ldots, N + 1$. As parameters to describe a function $v \in C_D^2(\bar{\Omega})$ we may choose the values $\eta_n = V(x_n)$ at the node points $n = 0, \ldots, N + 1$. To construct functions $v \in C_D^2(\bar{\Omega})$ we can define basis functions such that overlapping and neighboring hodes return

a value one and disconnected nodes return zero. Linear, quadratic, or any other valid basis functions can be used to formulate these functions over each interval. Here we define basis functions such that

$$\phi_j(x_i) = \begin{cases} 1 & \text{if} \quad i = j \\ 0 & \text{if} \quad i \neq j \end{cases} \quad i, j = 1, \ldots, N.$$

A function $v \in C_D^2(\bar{\Omega})$ then has the representation

$$v(x) = \sum_{m=1}^{M} \eta_m \phi_m(x) \quad x \in [0, 1].$$

Then the FEM can now be formulated as: Find $u_h \in C_D^2(\bar{\Omega})$ such that

$$a(u_h, v) = (f, v) \quad \forall v \in C_D^2(\bar{\Omega}).$$

If u_h satisfies the above then

$$a(u_h, \phi_m) = (f, \phi_m) \quad m = 1, \ldots, M \tag{*}$$

Then by taking linear combinations we see u_h satisfies ξ since

$$u_h(x) = \sum_{n=1}^{N} \xi_n \phi_n(x), \quad \xi_n = u_h(x_n)$$

rewrite * like so

$$\sum_{n=1}^{N} \xi_n a(\phi_n, \phi_m) = (f, \phi_m) \quad m = 1, \ldots, M$$

which is a linear system of equations with M equations and N unknowus $\xi_1, \ldots, \xi_N$ which we are solving for. In matrix form the linear system of equations can be written as

$$A\xi = b$$

Where A is the $M \times N$ matrix with elements $A_{mn} = a(\phi_n, \phi_m)$ called stiffness matrix. And where $\xi_n = (\xi_1, \ldots, \xi_N)$ and $b_m = (b_1, \ldots, b_M)$ with $b_m = (f, \phi_m)$ are both column vectors and b is the load vector. Row m is test and column n is trail.

1.3 General algorithm for stiffness matrix construction

Now we want to build a general algorithm to construct the stiffness matrix which decouples the geometric shape of the mesh from the functional degrees of freedom (the nodes). The algorithm will be general in the sense that any number of elements K can be chosen and the geometry, dimension, and basis type are separated through if/else statements in the code. Further calculus and Jacobian matrix is required if a general function for the construction of mesh element and nodes matrices is desired. Instead, the general framework advised by Prof He is to add additional definitions for linear VS quadratic basis functions or P_b and T_b information matrices into if/else statements. Inside the assembly code we will call these components (if/else statements) to construct the stiffness matrix.

The P matrix is an information matrix consisting of the coordinates of all nodes. It is a $D \times K$ matrix where D is dimension. In $1D$ and $2D$

$$P_{1D} = (x_0, \ldots, x_{K-1})$$
$$P_{2D} = \begin{pmatrix} x_0, \ldots, x_{K-1} \\ y_0, \ldots, y_{K-1} \end{pmatrix}$$

The T matrix is an information matrix containing the indices of the nodes of each element. Here the elements are the integral bounds over each interval. So the T matrix holds the indices of the elements E_k where k denotes the number of elements $k = 1, \ldots, K$. Define N_b to be the number of basis functions required to construct one element. Then T is a $N_b \times K$ matrix.

For the linear basis function case there are two basis functions per element so $N_b = 2$. Then the T matrix is given by

$$T = \begin{pmatrix} 1 & 2 & \cdots & K-1 & K \\ 2 & 3 & \cdots & K & K+1 \end{pmatrix}.$$

Only when we have linear basis functions we can define

contains Ω *points* $\mathbf{P} = \mathbf{P_b}$ *where b denotes basis node value.*
contains Ω *elements* $\mathbf{T} = \mathbf{T_b}$ *where b denotes basis element value.*

When our domain is defined with quadratic basis functions instead of linear then the number of points needed per element increases but NOT the

number of elements K in our domain. Then P_b will contain the midpoints in addition to the two endpoints to describe the quadratic curve. T and T_b then must also have an extra row so we know where this extra midpoint is physically located in terms of its neighbors. Thus P and T contain information about the geometry of the system which is represented in the element stiffness matrix. T_b contains algebraic information used to map the element stiffness matrix to the global stiffness matrix. P_b contains algebraic information about basis function construction. Then for quadratic basis $P \neq P_b$ and $T \neq T_b$.

Now consider the interval of a specific element $E = [x_n, x_{n+1}]$. Then because our basis functions are defined for $i \neq j$ such that $\phi_i(x_j) = 0$ and $\phi_i(x_i) = 1$, the only basis functions we consider over this element E are ϕ_n, ϕ_{n+1}. Note the following important property

$$(\phi_j, \phi_{j+1}) = (\phi_{j+1}, \phi_j).$$

Then we define local basis function ψ as ϕ over the interval E. Recall the linear basis functions:

$$\phi_n(x) = \begin{cases} \frac{x - x_{n-1}}{h} & \text{for } x \in [x_{n-1}, x_n] \\ \frac{x_{n+1} - x}{h} & \text{for } x \in [x_n, x_{n+1}] \\ 0 & \text{else} \end{cases}$$

Then define the local basis functions for an element $E = [x_n, x_{n+1}]$. On this element we have contributions from the positive edge of ϕ_{n+1} and negative from ϕ_n. Call the negative slope ψ_1 and the positive slope ψ_2 given by

$$\psi_1 = \frac{x_{n+1} - x}{h} \text{ for } x \in [x_n, x_{n+1}]$$
$$\psi_2 = \frac{x - x_n}{h} \text{ for } x \in [x_n, x_{n+1}].$$

So on one element E we have two local basis functions denoted $N_b = 2$ for linear global basis functions and the total number of global basis functions is K. We need two points $[x_n, x_{n+1}]$ to compute ψ_1, ψ_2 that are extracted directly from the P_b matrix using the indices from the T_b matrix:

$$x_n^{(k)} = P_b(1, T_b(1, k))$$
$$x_{n+1}^{(k)} = P_b(1, T_b(2, k))$$

There are four non-zero local integrals on $E_k = [x_n, x_{n+1}]$

$$a^k_{\beta\alpha} = a^k(\psi_\beta, \psi_\alpha)$$
$$a^k_{\beta\alpha} = \int_{x^k_n}^{x^k_{n+1}} c\left(\psi'_\beta \psi'_\alpha\right) dx \text{ for } \beta, \alpha \in \{1, 2\}$$

This means the local integral should be assembled to the global matrix entry a_{mn}. An element is given by $E_k = \left[x^k_n, x^k_{n+1}\right]$ for $k = 1, \ldots, K$. We compute an $N_b \times N_b$ element stiffness matrix a^k with indices $\alpha, \beta \in \{1, 2\}$ for the linear case. Because β is the test function index it indicates the rows of a^k and α the columns.

$$a^k_{\beta\alpha} = \begin{bmatrix} a^k_{11} & a^k_{12} \\ a^k_{21} & a^k_{22} \end{bmatrix} = \begin{bmatrix} \int c\psi'_1\psi'_1 & \int c\psi'_1\psi'_2 \\ \int c\psi'_2\psi'_1 & \int c\psi'_2\psi'_2 \end{bmatrix}$$

To map the local element stiffness matrix $a^k_{\beta\alpha}$ into the global $M \times K$ matrix A we use the topology matrix T_b to route the indices.

$$\text{Global row } m = T_b(\beta, k)$$
$$\text{Global column } n = T_b(\alpha, k)$$

I write the global entry of A as A_{mn} and the local element stiffness matrix as $a^k_{\beta\alpha}$. I include $\delta_{m,T_b(\beta,k)}$ as a way to search for the global row index and $\delta_n, T_b(\alpha, k)$ as a way to search for the global column index.

$$A_{mn} = \sum_{k=1}^{K} \sum_{\beta=1}^{N_b} \sum_{\alpha=1}^{N_b} a^k_{\beta\alpha} \cdot \delta_{m,T_b(\beta,k)} \cdot \delta_{n,T_b(\alpha,k)}$$

1.4 Neumann and Robin Boundary Conditions

When applying Neumann or Robin boundary conditions, the boundary terms resulting from Green's identity are non-zero and must be directly incorporated into the global linear system $A\xi = b$. Recall the boundary terms from the general weak formulation evaluated at the endpoints $x = a$ and $x = b$:

$$-c(b)u'(b)v(b) + c(a)u'(a)v(a) + \int_a^b cu'v'dx = \int_a^b fvdx$$

.

For a Neumann condition $u'(a) = r_a$ at the left boundary, substituting r_a and moving the known value to the right-hand side of the equation results in adding $-r_a c(a)$ to the first entry of the load vector b_0. Similarly, a Neumann condition $u'(b) = r_b$ at the right boundary results in adding $r_b c(b)$ to the final load vector entry b_{N+1}.

For a Robin boundary condition at the right boundary, defined as $u'(b) + q_b u(b) = p_b$, we substitute $u'(b) = p_b - q_b u(b)$ into the weak form. Rearranging the equation to group the unknown trial functions u on the left and the known source terms on the right yields:

$$q_b c(b)u(b)v(b) + \int_a^b cu'v'dx = \int_a^b fvdx + p_b c(b)v(b) - c(a)u'(a)v(a)$$

.

In the matrix formulation, because $q_b c(b)u(b)v(b)$ depends on the unknown u, we add $q_b c(b)$ to the corresponding diagonal entry of the global stiffness matrix $A_{N+1,N+1}$. The known term $p_b c(b)v(b)$ dictates that we add $p_b c(b)$ to the corresponding final entry of the load vector b_{N+1}.

Chapter 2

Programming

2.1 File structure

To program this we can utilize the following file structure:

```
FEMelliptic/
└── src/
    ├── main.py
    ├── build_stiffness_matrices.py
    ├── gaussian_quadrature.py
    ├── boundary_conditions.py
    ├── error.py
    ├── basis.py
    ├── information_matrices.py
    ├── functions.py
    └── documentation/
        └── FEM_basics.tex
```

The assembly code contains the general algorithm for stiffness matrix construction. The other components contains the if/else statements to define the geometry and basis functions. The main.py file is the driver code that calls the assembly code and the components to construct the stiffness matrix.

2.2 main.py

In main.py we will define the following parameters:

```
N = int # number of internal nodes in domain N which corresponds to the number of unknowns
K = N+1 # number of elements
N_b = int # number of basis functions per element
D = int # dimension of the problem
gauss_point_number = int # number of gauss points for integration

x = linspace(0, 1, N+2) # domain

stiffness_matrix = build_stiffness_matrix(
    N,
    N_b,
    K,
    P_b,
    T_b,
    c_func, # Stiffness matrix is computed with coeff function and load with driving function.
    basis_type,
    gauss_point_number)

load_matrix = build_load_matrix(
    N,
    N_b,
    K,
    P_b,
    T_b,
    f_func,
    basis_type,
    gauss_point_number)

left_boundary = functions.solution_func.hw_1(0.0) # homogeneous Dirichlet boundary condition
right_boundary = functions.solution_func.hw_1(1.0)
stiffness\_matrix, load\_matrix = apply\_dirichlet\_linear(stiffness\_matrix, load\_matrix, left\_boundary, right\_boundary) \\
```

```
32
33 analytical\_solution = functions.solution\_func(x) \\
34 FEM\_solution = np.linalg.solve(stiffness\_matrix, load\
      _matrix) \\
35 error = error(analytical\_solution.flatten(), FEM\
      _solution.flatten()) \\
```

2.3 build_stiffness_matrices.py

The *build_ stiffness_ matrices.py* file contains the general algorithms provided for A_{mn} and b_m. Here, instead of dirac delta functions we can use for loops to search for the row and column indices of the stiffness matrix.

```
1  fn build\_local\_stiffness\_matrix(c\_func, basis\_type,
      left\_node, right\_node, gauss\_point\_number, N\_b)
      \\
2      local\_stiffness\_matrix = zeros(N\_b, N\_b) \\
3      for beta in range(1, N\_b + 1): \\
4          for alpha in range(1, N\_b + 1): \\
5              fn integrand(x) \\
6                  d\_psi\_beta = basis\_type.derivative(
                       beta, left\_node, right\_node, x) \\
7                  d\_psi\_alpha = basis\_type.derivative(
                       alpha, left\_node, right\_node, x) \\
8                  return c\_func(x) * d\_psi\_beta * d\_psi
                       \_alpha \\
9              local\_stiffness\_matrix[beta - 1, alpha - 1]
                   = gauss\_integrate(
10                 integrand,
11                 left\_node,
12                 right\_node,
13                 gauss\_point\_number) \\
14     return local\_stiffness\_matrix # returns the local
          stiffness matrix for one element (2x2 for linear)
          \\
15
16
17 fn build\_stiffness\_matrix(N, N\_b, K, P\_b, T\_b, c\
      _func, basis\_type, gauss\_point\_number) \\
```

```
18      stiffness\_matrix = sparse(N+2, N+2) \\
19      for k in range(K): \\
20          left\_node = P\_b[0, T\_b[0, k]] \\
21          right\_node = P\_b[0, T\_b[1, k]] \\
22          local\_stiffness\_matrix = build\_local\
                _stiffness\_matrix(
23              c\_func,
24              basis\_type, # basis.linear or basis.
                    quadratic
25              left\_node,
26              right\_node,
27              gauss\_point\_number)\\
28          for beta in range(1, N\_b + 1): \\
29              for alpha in range(1, N\_b + 1): \\
30                  stiffness\_matrix[T\_b[beta - 1, k], T\_b
                        [alpha - 1, k]] += local\_stiffness\
                        _matrix[beta - 1, alpha - 1] \\
31      return stiffness\_matrix # returns the global
            stiffness matrix (NxN) \\
32
33  fn build\_load\_matrix(N, N\_b, K, P\_b, T\_b, f\_func,
        basis\_type, gauss\_point\_number) \\
34      load\_matrix = zeros(N+2, 1) \\
35      for k in range(K): \\
36          left\_node = P\_b[0, T\_b[0, k]] \\
37          right\_node = P\_b[0, T\_b[1, k]] \\
38          for beta in range(1, N\_b + 1): \\
39              fn integrand(x) \\
40                  psi\_beta = basis\_type.\_\_call\_\_(beta
                        , left\_node, right\_node, x)
41                  return f\_func(x) * psi\_beta \\
42              load\_matrix[T\_b[beta - 1, k]] += gauss\
                    _integrate(
43                  integrand,
44                  left\_node,
45                  right\_node,
46                  gauss\_point\_number
47              ) \\
48      return load\_matrix # returns the global load matrix
            (Nx1) \\
```

2.4 basis.py

File for basis functions basis.py contains the following functions:

```
class linear
    fn derivative(beta, left\_node, right\_node) \\
        h = right\_node - left\_node \\
        if beta == 1: \\
            return -1/h \\
        elif beta == 2: \\
            return 1/h \\
        else: \\
            raise ValueError("Invalid beta index for
               linear basis function") \\
    fn \_\_call\_\_(beta, left\_node, right\_node, x) \\
        h = right\_node - left\_node \\
        if beta == 1: \\
            return (right\_node - x) / h \\
        elif beta == 2: \\
            return (x - left\_node) / h \\
class quadratic
    fn derivative(beta, left\_node, right\_node, x) \\
        h = right\_node - left\_node \\
        if beta == 1: \\
            return (1/h) * (4 * (x - left\_node) - 3) \\
        elif beta == 2: \\
            return (1/h) * (4 * (x - left\_node) - 1) \\
        elif beta == 3: \\
            return (1/h) * (-8 * (x - left\_node) + 4) \\
        else: \\
            raise ValueError("Invalid beta index for
               quadratic basis function") \\
    fn \_\_call\_\_(beta, left\_node, right\_node, x) \\
        h = right\_node - left\_node \\
        if beta == 1: \\
            return 2 * ((x - left\_node)/h)**2 - 3 * ((x
               - left\_node)/h) + 1
        if beta == 2: \\
            return 2 * ((x - left\_node)/h)**2 - ((x -
               left\_node)/h)
        if beta == 3: \\
```

```
34              return -4 * ((x - left\_node)/h)**2 + ((x -
                   left\_node)/h)
```

2.5 gaussian_quadrature.py

File for guassian quadrature gaussian_quadrature.py contains the following functions:

```
1  fn generate\_gauss\_reference\_1D(gauss\_point\_number)
2      if gauss\_point\_number == 2:
3          gauss\_coefficient\_reference\_1D = [1.0, 1.0] \\
4          gauss\_point\_reference\_1D = [-1.0/sqrt(3), 1.0/
               sqrt(3)]
5      elif gauss\_point\_number == 4:
6          gauss\_coefficient\_reference\_1D =
               [0.3478548451,
7          0.3478548451, 0.6521451549, 0.6521451549] \\
8          gauss\_point\_reference\_1D = [0.8611363116,
9          -0.8611363116, 0.3399810436, -0.3399810436] \\
10     elif gauss\_point\_number == 8:
11         gauss\_coefficient\_reference\_1D =
               [0.1012285363,
12         0.1012285363, 0.2223810345, 0.2223810345,
               0.3137066459,
13         0.3137066459, 0.3626837834, 0.3626837834] \\
14         gauss\_point\_reference\_1D = [0.9602898565,
15         -0.9602898565, 0.7966664774, -0.7966664774,
               0.5255324099,
16         -0.5255324099, 0.1834346425, -0.1834346425] \\
17     else:
18         raise ValueError("Unsupported number of Gauss
               points. Choose 2, 4, or 8.")
19     return gauss\_coefficient\_reference\_1D, gauss\
           _point\_reference\_1D
20
21 fn generate\_gauss\_local\_1D(gauss\_coefficient\
       _reference\_1D, gauss\_point\_reference\_1D, lower\
       _bound, upper\_bound)
22     gauss\_coefficient\_local\_1D = []
```

```
23      gauss\_point\_local\_1D = []
24
25      for i in range(len(gauss\_coefficient\_reference\_1D)
           )
26          local\_coeff = (upper\_bound - lower\_bound) *
               gauss\_coefficient\_reference\_1D[i] / 2.0
27          local\_point = (upper\_bound - lower\_bound) *
               gauss\_point\_reference\_1D[i] / 2.0 + (upper\
               _bound + lower\_bound) / 2.0
28          gauss\_coefficient\_local\_1D.append(local\_coeff
               )
29          gauss\_point\_local\_1D.append(local\_point)
30      return gauss\_coefficient\_local\_1D, gauss\_point\
           _local\_1D
31
32  fn gauss\_integrate(integrand\_func, left\_node, right\
       _node, gauss\_point\_number)
33      ref\_coeffs, ref\_points = generate\_gauss\_reference
           \_1D(gauss\_point\_number)
34      local\_coeffs, local\_points = generate\_gauss\_local
           \_1D(ref\_coeffs, ref\_points, left\_node, right\
           _node)
35      integral = 0.0
36      for i in range(gauss\_point\_number)
37          integral += local\_coeffs[i] * integrand\_func(
               local\_points[i])
38      return integral
```

2.6 information_matrices.py

File for generating the P, T, P_b, and T_b information matrices for linear and quadratic 1D. The information matrices are stored in information_matrices.py and contain the following functions:

```
1   class linear
2       fn P(N)
3           return np.array([np.linspace(0, 1, N+2)]) #
               returns the P matrix for 1D linear basis
               functions
```

```
    fn T(N)
        K = N + 1
        return np.array([
            [k for k in range(K)],
            [k + 1 for k in range(K)]
        ]) # returns the T matrix for 1D linear basis
           functions
    fn P\_b\_linear(N)
        return linear.P(N)
    fn T\_b\_linear(N)
        return linear.T(N)

class quadratic
    # Implement similar functions for quadratic basis
       functions
```

2.7 functions.py

functions.py contains the c_funct and f_func functions:

```
class c\_func
    fn hw\_1(x)
        return exp(x)
class f\_func
    fn hw\_1(x)
        return -exp(x) * (cos(x) - 2 * sin(x) - x * cos(x
           ) - x * sin(x))
class solution\_func(x)
    return x * cos(x)
```

2.8 boundary_conditions.py and error.py

The boundary_conditions.py file contains the following functions:

```
class dirichlet
    fn apply\_dirichlet\_linear(stiffness\_matrix, load\
       _matrix, left\_boundary, right\_boundary)
        stiffness\_matrix[0, :] = 0
```

```
        stiffness\_matrix[0, 0] = 1
        load\_matrix[0] = left\_boundary

        stiffness\_matrix[-1, :] = 0
        stiffness\_matrix[-1, -1] = 1
        load\_matrix[-1] = right\_boundary

        return stiffness\_matrix, load\_matrix

class robin
    fn apply\_robin\_linear(stiffness\_matrix, load\
       _matrix)
        pass # Implement the Robin boundary condition
           logic here
```

The error.py file contains the following functions:

```
fn error(analytical\_solution, FEM\_solution):
    abs\_max\_difference = np.max(np.abs(analytical\
       _solution - FEM\_solution))
    return abs\_max\_difference
```

NOTATION KEY

- N = number of internal nodes in domain N which corresponds to the # of cukhowns
- $k = G$ lobal element index $(k = 1, \ldots, N-1)k$
- m = Global test basis index $(1, \ldots, M)$ maps rows of AM
- n = Global trail basis index $(1, \ldots, N)$ maps to columns of An
- β = local test basis index $\beta \in \{1, 2\}$ beta
- α = local trail basis index $\alpha \in \{1, 2\}$ alpha
- ξ_n = global trial coefficients x_i
- η_m = global test coefficients eta
- $a^k_{\beta\alpha}$ = local stiffness matrix local-a
- domain = $X = x_0, \ldots, x_{N+1} \quad x$
- T, T_b = topology information matrix T-matrix, T - b-matrix
- P, P_b = index information matrix P-matrix, P - b-matrix